

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTY OF ENGINEERING
SYSTEMS ENGINEERING

Dashboard Implementation Report

Workshop No. 4

Course: Data Analysis Programming

Authors:

Juan Diego Grajales Castillo
Carmen Sofía Flórez Juajibioy
Edgar Alejandro Mora Chala

Bogotá, Colombia
May 30, 2026

Contents

1	Executive Summary	2
2	Pipeline Integration Overview	2
3	Gold Layer Architecture	2
4	Governance Dashboard	3
4.1	Design Rationale	3
4.2	Component Inventory	3
4.3	KPI Mapping	3
5	Storytelling Dashboard	3
5.1	Narrative Design	4
5.2	Component Inventory	4
5.3	User Story Mapping	4
6	Visualization Design Decisions	5
7	DAG Task Graph	5
8	Integration Adjustments	6
9	Reproducibility & Setup	7
10	Challenges and Resolutions	8

1 Executive Summary

This report documents the design, implementation, and integration of two independent Plotly Dash web applications that serve as the presentation layer of a Medallion data pipeline processing public sentiment on consumer technology products. The pipeline ingests data from two complementary sources: the Twitter/X API (real-time consumer opinions) and GSM Arena / ArenaEV web scraping (structured tech news comments). Both data streams are processed through Bronze, Silver, and Gold layers orchestrated by Apache Airflow on Docker.

The Governance Dashboard (localhost:8050) targets data engineers and analysts, surfacing data quality KPIs, null rates, volume trends, and schema compliance from the Gold governance parquet files. The Storytelling Dashboard (localhost:8051) targets functional business users, translating sentiment scores, brand rankings, and topic trends into plain-language narratives and intuitive visualizations.

- 8 Gold Artifacts
- 2 Data Sources
- 2 Dashboard Apps
- 8 DAG Tasks

2 Pipeline Integration Overview

The full pipeline consists of three sequential DAG stages, each triggered after the previous layer completes successfully. A FileSensor in the Silver DAG watches for new Bronze JSON files before initiating processing, ensuring no stale data propagates.

Layer	DAG ID	Output Format	Trigger
Bronze	bronze.ingestion	JSON (raw)	Manual/Scheduled
Silver	silver_twitter, silver_webscraping	sil- Parquet (cleaned)	FileSensor on Bronze
Gold	gold_sentiment_weekly	Parquet (8 artifacts)	TriggerDagRunOperator

Table 1: Pipeline Integration Overview

3 Gold Layer Architecture

The Gold DAG (gold_sentiment_weekly) produces eight distinct artifacts, organized into two functional groups: Governance (data quality traceability) and Analytics (business intelligence). Governance artifacts are stored in dedicated subfolders to maintain clear separation of concerns.

Taxonomy coverage. Brand, product, and topic enrichment is driven by keyword dictionaries aligned to the three X API queries used in Bronze ingestion (PhonesQuery,

#	Artifact	Location	Format
1	governance_tweets_weekly	gold/governance/tweets/	Parquet
2	governance_news_weekly	gold/governance/news/	Parquet
3	brand_analytics_weekly	gold/	Parquet
4	topic_analytics_weekly	gold/	Parquet
5	product_analytics_weekly	gold/	Parquet
6	top_influencers_weekly	gold/	Parquet
7	representative_comments_weekly	gold/	Parquet
8	storytelling_weekly	gold/	Parquet

Table 2: Gold Layer Artifacts

Computers Query, Accessories Query), ensuring that only terms that can realistically appear in the Silver data generate taxonomy matches. Extended topics include foldable, ai, ev, and smartwatch to capture GSM Arena and ArenaEV content patterns.

4 Governance Dashboard

File: dashboard/governance_app.py — **Port:** localhost:8050 — **Audience:** Data Engineers / Analysts

4.1 Design Rationale

The governance dashboard is designed around a single question: can I trust the data in the Gold layer? The layout follows a progressive disclosure pattern: summary KPI cards at the top provide an immediate health signal, followed by progressively more detailed charts (null rates, volume trends, outlier distributions) that allow analysts to drill into specific quality issues.

Data is loaded exclusively from the Gold governance parquet files using `pd.read_parquet()`. The application polls for the latest file on every page load, so it always reflects the most recent DAG run without requiring a manual refresh.

4.2 Component Inventory

4.3 KPI Mapping

The following table maps each dashboard KPI to its source column in the governance parquet, the computation method, and the alerting threshold used to color-code the metric card.

5 Storytelling Dashboard

File: dashboard/storytelling_app.py — **Port:** localhost:8051 — **Audience:** Functional / Business Users

Component	Type	Description
KPI Summary Cards	Metric Cards	Four numeric cards: total records ingested, overall null rate (%), duplicate rate (%), and schema compliance rate (%). Color-coded green/yellow/red by threshold.
Null Rate Bar Chart	Bar (horiz.)	Horizontal bar chart showing null rate per field, ordered descending by severity. Fields above the 5% threshold rendered in red. Source annotation included.
Volume Over Time	Line/Bar	Weekly record volume per source (Twitter vs. GSM Arena). Dual-axis where needed. Highlights ingestion peaks and data gaps.
Outlier Rate Chart	Bar (horiz.)	Outlier rate per numeric field (likeCount, replyCount, follower count) using IQR method. Helps identify data spikes from viral content.
Data Quality Table	Table	Full tabular view of all KPIs with current values, defined thresholds, and pass/fail status. Color-coded rows. Exportable via Dash DataTable.
Header Section	Static	Project title, team name, topic angle, data last-updated timestamp pulled from parquet file meta-data.

Table 3: Governance Component Inventory

5.1 Narrative Design

The storytelling dashboard follows a top-down narrative arc: it opens with the executive summary text pulled directly from the storytelling_weekly parquet, then guides the user through brand rankings, topic sentiment, product performance, and representative community comments. Every chart label uses plain language - no column names, no technical jargon.

The interface targets the functional user persona defined in Workshop #1: a product manager or marketing analyst who wants to understand what people are saying about tech products this week without having to interpret raw numbers. Charts prioritize insight over data density, and each section is preceded by a one-sentence plain-language interpretation.

5.2 Component Inventory

5.3 User Story Mapping

Each dashboard component maps to a specific user question that the functional user would ask when opening the dashboard on a Monday morning.

KPI	Source Column	Computation	Alert Threshold
Total Records	tweet_id / newsLink	COUNT(*)	-
Null Rate (%)	All columns	$(\text{null_count} / \text{total_rows}) \times 100$	$> 5\% \rightarrow \text{red}$
Duplicate Rate (%)	tweet_id / newsLink	$(\text{dup_count} / \text{total_rows}) \times 100$	$> 2\% \rightarrow \text{yellow}$
Schema Compliance	Column presence	$(\text{present_cols} / \text{expected_cols}) \times 100$	$< 95\% \rightarrow \text{red}$
Avg Sentiment	sentiment_score	MEAN(sentiment_score)-	
Outlier Rate (%)	likeCount, followers	IQR method per field	$> 10\% \rightarrow \text{yellow}$

Table 4: KPI Mapping Details

6 Visualization Design Decisions

Color Palette: A single colorblind-friendly palette is shared across both dashboards. Sentiment encoding is fixed and never reused for other dimensions to avoid cognitive overload.

Chart Type Rationale:

- **Horizontal bar charts:** Used for all ranking comparisons because they handle long labels naturally and support left-to-right reading of magnitude.
- **Line charts for time series:** Sentiment trends are displayed as lines, not bars, because the continuity of a line better conveys opinion shifts over time.
- **Donut over pie chart:** Allows a central label showing the dominant sentiment, improving scannability.
- **Stacked bars for breakdown:** Shows volume + sentiment split simultaneously without requiring two separate charts.
- **Tables for governance KPIs:** Analysts need exact numbers and threshold comparisons, not visual approximations.

No more than 5 distinct chart types are used across both dashboards. Consistency reduces the cognitive cost of learning the interface.

7 DAG Task Graph

The Gold DAG runs every Sunday at midnight (ISO week boundary). Tasks 1 and 2 (governance) run in parallel as they read from independent Silver sources. Tasks 3-7 (analytics) depend on both governance tasks completing successfully. Task 8 (storytelling)

Component	Type	Description
Narrative Summary	Text Card	Auto-generated executive summary pulled from the storytelling parquet. Highlights top brand, best-received brand, most-discussed topics, and top products.
Sentiment Dist.	Donut Chart	Overall breakdown of positive/negative/neutral records across all sources. Color encoding: green=positive, red=negative, grey=neutral.
Sentiment Trend	Line Chart	Average sentiment score over time (weekly). Separate lines for Twitter and News sources.
Brand Ranking	Bar (horiz.)	Total mentions per brand, color-coded by dominant sentiment.
Top Products	Bar Chart	Mention volume per product (iphone, galaxy, rtx, etc.) with sentiment breakdown stacked.
Topic Sentiment	Bar + Color	Topics ranked by mention volume; bar color encodes net sentiment.
Source Comparison	Side-by-Side	Volume and sentiment comparison between Twitter API and GSM Arena scraping.
Rep. Comments	Quote Cards	Best and worst community comment per brand.
Header Section	Static	Project title, week number, and data last-updated timestamp.

Table 5: Storytelling Component Inventory

acts as a final aggregation step, reading the outputs of all preceding analytics tasks. Parallelism reduces total DAG wall-clock time by approximately 60% compared to sequential execution.

8 Integration Adjustments

- **Unified governance folder structure:** Governance artifacts are now written to separate subfolders (`gold/governance/tweets/` and `gold/governance/news/`) to prevent file glob collisions.
- **Storytelling artifact converted to Parquet:** Converted from JSON to a single-row Parquet file to maintain consistency and allow `pd.read_parquet()` loading.
- **Nested columns serialized before Spark write:** Python lists or dicts are serialized to JSON strings before Spark DataFrame creation to avoid schema inference failures in local mode.
- **Extended topic taxonomy:** Added ‘foldable’, ‘ai’, ‘ev’, and ‘smartwatch’ to cover GSM Arena terms; resolving a 30% null topic issue.
- **Comment noise filter:** Comments shorter than 20 characters are filtered to prevent single-word noise appearing as representative insights.

User Question	Component	Gold Source
What's the overall mood about tech this week?	Sentiment Donut	storytelling_weekly
Which brand dominated the conversation?	Brand Ranking Bar	brand_analytics_weekly
Is sentiment improving or getting worse?	Sentiment Trend Line	brand_analytics_weekly
What products are people actually talking about?	Top Products Bar	product_analytics_weekly
What topics are driving positive/negative reactions?	Topic Heatmap	topic_analytics_weekly
What are real users actually saying about Samsung?	Comment Cards	representative_comments_weekly
Is Twitter more positive than news site comments?	Source Comparison	gov_tweets / gov_news
What's the one thing I should take away from this week?	Narrative Summary	storytelling_weekly

Table 6: User Story Mapping

Color	Hex	Semantic Use
Green	#2ECC71	Positive sentiment - KPI cards in healthy state
Red	#E74C3C	Negative sentiment - KPI alerts above threshold
Grey	#7F8C8D	Neutral sentiment - secondary information
Navy	#0D1B2A	Headers, primary brand identity
Cyan	#00C2CB	Accent - links, highlights, dividers
Blue	#1B4FBF	Interactive elements, selected states, chart primary series

Table 7: Color Palette

- **Spark session stop() per task:** Explicitly stopping the `SparkSession` avoids conflicts when Airflow runs multiple Python tasks in the same worker process.

9 Reproducibility & Setup

All services run inside Docker; no local Python environment is required beyond Docker itself. The complete project can be reproduced with the following steps:

1. **Clone & configure:** `git clone && cd cp .env.example .env`
2. **Start Airflow:** `docker-compose up -d`
3. **Trigger DAGs in sequence:** Trigger `bronze_ingestion` → `silver_twitter` + `silver_web scraping` → `gold_sentiment_weekly`.
4. **Launch Governance Dashboard:** `cd dashboard && python governance_app.py`

Task Name	Inputs	Description
governance_tweets	(reads Silver)	Enrich tweets with taxonomy. Run VADER sentiment.
governance_news	(reads Silver)	Explode news comments. Enrich with taxonomy. Run VADER.
brand_analytics	gov_tweets + gov_news	Merge sources by brand. Compute counts, sentiment %.
topic_analytics	gov_tweets + gov_news	Aggregate by topic across sources. Compute sentiment %.
product_analytics	gov_tweets + gov_news	Aggregate by product. Infer brand. Sentiment breakdown.
top_influencers	gov_tweets only	Group by author + brand. Rank by engagement.
representative_comments	gov_tweets + gov_news	Select highest/lowest-scoring comment per brand.
storytelling	All 5 analytics tasks	Synthesize artifacts into single-row summary parquet.

Table 8: DAG Task Workflow

5. **Launch Storytelling Dashboard:** `cd dashboard && python storytelling_app.py`

Dashboard code lives under `dashboard/`. The repository is tagged `v1.0-workshop4` at the delivery commit.

10 Challenges and Resolutions

- **PySpark nested type serialization (Severity: HIGH):** Spark’s local mode cannot infer schemas for Python dict/list columns. **Resolution:** Serialize all complex columns to JSON strings before `spark.createDataFrame()` and parse them back in the dashboard.
- **GSM Arena comment noise (Severity: MEDIUM):** Comments contained metadata mixed with text. **Resolution:** Applied a 20-character minimum text filter and excluded strings matching known metadata patterns.
- **Snapshot date parsing heterogeneity (Severity: MEDIUM):** Contains both Unix timestamps and ISO dates. **Resolution:** A Spark conditional expression detects numeric strings to apply `from_unixtime()` or falls back to `substring`.
- **SparkContext conflicts in Airflow (Severity: HIGH):** Running multiple tasks caused already stopped errors. **Resolution:** Ensure each task fully isolates and stops its own `SparkSession`.
- **Dashboard data freshness (Severity: LOW):** Plotly Dash does not natively poll for new files. **Resolution:** A `dcc.Interval` component triggers a 5-minute callback to re-scan the Gold directory and reload if a newer file is found.

References

- [1] Plotly Dash Developer Documentation. *Dash Architecture and Component Reference*. Disponible en: <https://dash.plotly.com/>
- [2] Apache Software Foundation. *Apache Airflow Documentation: Orchestrating Complex Data Pipelines*. Disponible en: <https://airflow.apache.org/>
- [3] Databricks Engineering. *What is the Medallion Design Pattern? Bronze, Silver, and Gold Data Layers*. Disponible en: <https://www.databricks.com/glossary/medallion-architecture>
- [4] Apache Spark. *PySpark SQL and DataFrames API Reference*. Disponible en: <https://spark.apache.org/docs/latest/api/python/>
- [5] Hutto, C.J. & Gilbert, E.E. (2014). *VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text*. Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14).
- [6] Docker Documentation. *Multi-container Orchestration with Docker Compose*. Disponible en: <https://docs.docker.com/compose/>